

بخش اول پروژه

نظریه زبان‌ها و ماشین‌ها

دکتر انتظاری ملکی - دکتر باغبانی

طراحان: میلاد زارعی ملکی - طاها بیکلریان

ترم ۴۰۴۲



دانشگاه علم و صنعت ایران

فهرست مطالب

۲	پیاده‌سازی ماشین تورینگ و بیدستر پرتلاش (Busy Beaver)
۲	ساختار یکپارچه کلاس ماشین تورینگ (turing_machine.py)
۳	بخش اول - شبیه‌ساز ماشین تورینگ
۴	بخش دوم - محاسبات ریاضی ماشین تورینگ
۴	بخش سوم - بیدستر پرتلاش (Busy Beaver)
۴	برنامه ۲ کارتی بیدستر پرتلاش نمونه
۵	پیوست
۵	مراجع

پیاده‌سازی ماشین تورینگ و بیدستر پرتلاش (Busy Beaver)

ماشین تورینگ یک نمایش حالت‌مبنا از محاسبات است که در عمل، ساده‌ترین شکل یک کامپیوتر را ارائه می‌دهد. یکی از مؤلفه‌های کلیدی ماشین تورینگ، قوانین انتقال (transition rules) هستند که الگوریتم قابل اجرای ماشین روی نوار را پیاده‌سازی و رمزگذاری می‌کنند.

در این بخش پروژه، شما پیاده‌سازی یک شبیه‌ساز کامل ماشین تورینگ را در فایل `tur-ing_machine.py` بر اساس یک ساختار کلاس یکپارچه که در ادامه آمده است انجام خواهید داد. شبیه‌ساز شما در ابتدا باید به صورت یک طرفه نامتناهی (فقط از راست) کار کند و در ادامه در بخش سوم آن را به یک نوار نامتناهی دوطرفه گسترش دهید. پیاده‌سازی شبیه‌ساز با استفاده از ژنراتورهای پایتون (generators) طراحی می‌شود که خروجی را گام‌به‌گام تولید می‌کند. شما همچنین قادر خواهید بود پیکربندی ماشین را در حین محاسبات چاپ کنید تا بتوانید الگوریتم‌های خواسته شده در بخش‌های بعدی پروژه را اشکال‌زدایی (debug) و پیاده‌سازی نمایید.

شما می‌توانید صحت پیاده‌سازی خود را با اسکریپت خودآزمایی `student_test_turing.py` بررسی کنید.

ساختار یکپارچه کلاس ماشین تورینگ (`turing_machine.py`)

برای یکپارچگی محاسبات و امکان اجرای تست‌های خودکار، کلاس ماشین تورینگ شما باید دقیقاً مطابق ساختار و امضاهای زیر پیاده‌سازی شود:

```
1 class TuringMachine:
2     def __init__(self, transitions, start_state='q0',
3         ↪ accept_state='qa', reject_state='qr', blank_symbol=''):
4         """
5         Initialize the Turing machine.
6         transitions: transition dictionary with format of:
7         ↪ {(state, symbol): (next_state, write_symbol, direction)}
8         """
9         pass
10
11     def run(self, input_):
12         """
13         Run the simulator as a Python Generator.
14         This method should yield a tuple for each computational
15         ↪ step: (action, configuration).
16         action: the value 'Accept' or 'Reject' or None
17         configuration: a dictionary in the format:
18         ↪ {
19             'state': state,
20             'left_hand_side': list_of_symbols_on_left, # in
21             ↪ reverse order (the symbol closest to the head is last)
22             'symbol': current_symbol_under_head,
```

```

۱۹         'right_hand_side': list_of_symbols_on_right
۲۰     }
۲۱     """
۲۲     pass
۲۳
۲۴     def accepts(self, input_, step_limit=100):
۲۵         """
۲۶         Check whether the input string is accepted.
۲۷         Simulates up to step_limit steps.
۲۸         Returns: True if accepted, False if rejected, and None
۲۹         ↪ if the step limit is reached without halting.
۳۰         """
۳۱         pass
۳۲
۳۳     def rejects(self, input_, **kwargs):
۳۴         """
۳۵         Check whether the input string is rejected.
۳۶         The return value is the opposite of accepts.
۳۷         """
۳۸         pass
۳۹
۴۰     def debug(self, input_, step_limit=100, colored=False):
۴۱         """
۴۲         Print the machine configuration step by step on the
۴۳         ↪ terminal for debugging.
۴۴         The output format should be: state_name
۴۵         ↪ left[symbol]right.
۴۶         """
۴۷         pass

```

بخش اول – شبیه‌ساز ماشین تورینگ

پس از پیاده‌سازی کلاس فوق در فایل `turing_machine.py` و تایید آن با تست خودارزیابی، مجموعه‌ای از اسکریپت‌های نمونه برای ارائه مثال‌های برنامه‌نویسی از شبیه‌ساز در نظر گرفته شده است. برنامه‌های تست زیر را اجرا کرده و خروجی‌ها را بررسی نمایید.

نکته: اسکریپت‌های `test_turing_machine_example1.py` و `test_turing_machine_example2.py` را ببینید.

الف) حالت‌های (`states`) هر یک از ماشین‌های تورینگ ارائه‌شده در اسکریپت‌های تست فوق را فهرست کنید.

ب) دستور اجرا را در اسکریپت‌ها به گونه‌ای تغییر دهید که بتوان پیکربندی (`configuration`) ماشین را به ازای هر انتقال (`transition`) مشاهده کرد.

ج) توصیف کنید که هر یک از ماشین‌های تورینگ نمونه، چه محاسباتی را انجام می‌دهند.

بخش دوم – محاسبات ریاضی ماشین تورینگ

یک ماشین تورینگ می‌تواند هر نوع محاسباتی، از جمله عملیات‌های ریاضی را نمایش دهد. ما پیش از پرداختن به محاسبات کلی‌تر، چند ماشین پایه‌ای برای جمع و ضرب یکانی پیاده‌سازی خواهیم کرد.

نکته: اسکریپت‌های `test_turing_adder.py` و `test_turing_multiplier.py` را ببینید.

الف) یک ماشین تورینگ بسازید که عمل جمع دو عدد را با استفاده از نمایش یکانی (unary representation) محاسبه کند. می‌توانید فرض کنید اعداد با کاراکتر 0 از هم جدا شده‌اند و نماد خالی (blank symbol) رشته خالی است. نشان دهید که ماشین می‌تواند ورودی‌های زیر را محاسبه کند:

۱. $2+3$ به صورت 110111 که خروجی نهایی نوار آن باید 11111 باشد.

۲. $3+4$ به صورت 11101111 که خروجی نهایی نوار آن باید 1111111 باشد.

ب) یک ماشین تورینگ بسازید که عمل ضرب دو عدد را با استفاده از نمایش یکانی محاسبه کند. می‌توانید فرض کنید اعداد با کاراکتر 0 از هم جدا شده‌اند و نماد خالی، رشته خالی است. در صورت نیاز می‌توانید از نمادهای نوار اضافی استفاده کنید. نشان دهید که ماشین می‌تواند ورودی‌های زیر را محاسبه کند:

۱. $2*3$ به صورت 110111 که خروجی نهایی نوار آن باید 111111 باشد.

۲. $3*5$ به صورت 111011111 که خروجی نهایی نوار آن باید 11111111111111 باشد.

بخش سوم – بیدستر پرتلاش (Busy Beaver)

بازی بیدستر پرتلاش (پیوندهای موجود در پیوست را ببینید) عبارت است از ساخت یک ماشین تورینگ متوقف‌شونده با الفبای دودویی که بیشترین تعداد نماد 1 را با استفاده از تعداد محدودی از حالت‌ها (یا کارت‌ها) به همراه یک حالت پذیرش یا توقف (halt state) روی نوار خود بنویسد. هر کارت معمولاً نشان‌دهنده یک حالت و انتقال‌های آن بر اساس کاراکتر الفبای مواجه‌شده است. نمونه زیر یک برنامه ۲ کارتی است:

برای

برنامه ۲ کارتی بیدستر پرتلاش نمونه

Card A - 0: B1R - 1: B1L

Card B - 0: A1L - 1: H1R

نکته: اسکریپت های busy_beaaver.py و test_busy_beaaver_2.py را ببینید.

الف) توضیح دهید چرا پیدا کردن برنامه های بیدستر پرتلاش کار ساده ای نیست و چرا این مسئله به عنوان یک مسئله غیرقابل حل (undecidable) شناخته می شود.

ب) پیاده سازی نوار یک طرفه نامتناهی در شبیه ساز شما هنگام حرکت به سمت چپ از اولین خانه نوار با هشدار یا خطا متوقف می شود. ماژول turing_machine.py خود را به گونه ای تغییر دهید که شبیه ساز ماشین تورینگ اکنون از یک نوار نامتناهی دوطرفه (چپ و راست) پشتیبانی کند تا در زمان حرکت به چپ از مرز نوار، نوار به طور داینامیک گسترش یابد.

ج) چرا استفاده از ژنراتورهای پایتون (Python generators) در شبیه سازی ماشین های تورینگ سودمند است؟

د) شبیه ساز ماشین تورینگ خود را برای اجرای برنامه ۲ کارتی بیدستر پرتلاش فوق برنامه نویسی کنید. چه تعداد نماد 1 به دست می آورید؟

ه) این نتیجه را با برنامه های ۲ کارتی شناخته شده مقایسه کنید؛ خروجی شما چگونه ارزیابی می شود؟

و) برنامه های ۳ کارتی و ۴ کارتی بیدستر پرتلاش خود را بنویسید و تعداد نمادهای 1 که برنامه شما می نویسد را نشان دهید. توجه کنید که الزاما نباید بهترین برنامه های ۳ کارتی و ۴ کارتی را پیدا کنید، بلکه فقط برنامه هایی که بیشترین تعداد نماد 1 را نسبت به برنامه های ۲ کارتی می نویسند، کافی است.

م) آیا می توانید یک برنامه بیدستر پرتلاش ۵ کارتی جدید پیدا کنید؟ توجیه کنید که چرا فکر می کنید برنامه ۵ کارتی شما جدید است و یا نشان دهید که متوقف می شود یا دلایلی برای چرایی توقف آن ارائه دهید. توجه کنید که الزاما نباید بهترین برنامه ۵ کارتی را پیدا کنید، بلکه فقط برنامه ای که بیشترین تعداد نماد 1 را نسبت به برنامه های ۲، ۳ و ۴ کارتی می نویسد، کافی است.

ک) به سایت Busy Beaver Wiki مراجعه کنید و برنامه های بیدستر پرتلاش ۲، ۳ و ۴ کارتی را بررسی کنید. بهترین برنامه های ۲، ۳ و ۴ کارتی را با برنامه های خود مقایسه کنید. آیا برنامه های شما بهتر از بهترین برنامه های شناخته شده هستند؟ بهترین برنامه های شناخته شده را در برنامه های خود پیاده سازی کنید و نتایج را با برنامه های خود مقایسه کنید.

پیوست

- YouTube Video: Turing Machine Primer - Computerphile
- YouTube Video: Busy Beaver Turing Machines - Computerphile

مراجع

- Moore, C., Mertens, S., 2011. *The Nature Of Computation*. Oxford University Press.